

Agenda

- Multi-Threading

Platform Independence

- Java is architecture neutral i.e. can work on various CPU architectures like x86, ARM, SPARC, PPC, etc (if JVM is available on those architectures).
- Java is NOT fully platform independent. It can work on various platforms like Windows, Linux, Mac, UNIX, etc (if JVM is available on those platforms).
- Few features of Java remains platform dependent.
 - Multi-threading (Scheduling, Priority)
 - File IO (Performance, File types, Paths)
 - AWT GUI (Look & Feel)
 - Networking (Socket connection)

Program

- Program is set of instructions given to the computer.
- Executable file is a program.
- Executable file contains text, data, rodata, symbol table, exe header.

Process

- Process is program in execution.
- Program (executable file) is loaded in RAM (from disk) for execution. Also OS keep information required for execution of the program in a struct called PCB (Process Control Block).
- Process contains text, data, rodata, stack, and heap section.

Thread

- Threads are used to do multiple tasks concurrently within a single process.
- Thread is a lightweight process.
- When a new thread is created, a new TCB is created along with a new stack. Remaining sections are shared with parent process.

Process vs Thread

- Process is a container that holds resources required for execution and thread is unit of execution/scheduling.
- Each process have one thread created by default -- called as main thread.

Multi-threading (Java)

- Java applications are always multi-threaded.
- When any java application is executed, JVM creates (at least) two threads.
 - main thread -- executes the application main()
 - GC thread -- does garbage collection (release unreferenced objects)

- Programmer may create additional threads, if required.

Thread creation

- To create a thread
 - step 1: Implement a thread function (task to be done by the thread)
 - step 2: Create a thread (with above function)
- Method 1: extends Thread

```
class MyThread extends Thread {  
    @Override  
    public void run() {  
        // task to be done by the thread  
    }  
}
```

```
MyThread th = new MyThread();  
th.start();
```

- Method 2: implements Runnable

```
class MyRunnable implements Runnable {  
    @Override  
    public void run() {  
        // task to be done by the thread  
    }  
}
```

```
MyRunnable runnable = new MyRunnable();  
Thread th = new Thread(runnable);  
th.start();
```

- Java doesn't support multiple inheritance. If your class is already inherited from a super class, you cannot extend it from Thread class. Prefer Runnable in this case; otherwise you may choose any method.

```
// In Java GUI application is inherited from Frame class.  
// to create run() in the same class, you must use Runnable  
class MyGuiApplication extends Frame implements Runnable {  
    // ...  
    public void run() {  
        // ...  
    }  
}
```

```
// ...  
}
```

Daemon threads

- By default all threads are non-daemon threads (including main thread).
- We can make a thread as daemon by calling its `setDaemon(true)` method -- before starting the thread.
- Daemon threads are also called as background threads and they support/help the non-daemon threads.
- When all non-daemon threads are terminated, the Daemon threads get automatically terminated.

Thread life cycle

- `Thread.State state = th.getState();`
- NEW, RUNNABLE, BLOCKED, WAITING, TIMED_WAITING, TERMINATED
 - NEW: New thread object created (not yet started its execution).
 - RUNNABLE: Thread is running on CPU or ready for execution. Scheduler picks ready thread and dispatch it on CPU.
 - BLOCKED: Thread is waiting for lock to be released. Thread blocks due to synchronized block/method.
 - WAITING: Thread is waiting for the notification. Waiting thread release the acquired lock.
 - TIMED_WAITING: Thread is waiting for the notification or timeout duration. Waiting thread release the acquired lock.
 - TERMINATED: Thread terminates when `run()` method is completed, stopped explicitly using `stop()`, or an exception is raised while executing `run()`.

Synchronization

- When multiple threads try to access same resource at the same time, it is called as Race condition.
- Example: Same bank account undergo `deposit()` and `withdraw()` operations simultaneously.
- It may yield in unexpected/undesired results.
- This problem can be solved by Synchronization.
- The `synchronized` keyword in Java provides thread-safe access.
- Java synchronization internally use the Monitor object associated with any object. It provides lock/unlock mechanism.
- "`synchronized`" can be used for block or method.
- It acquires lock on associated object at the start of block/method and release at the end. If lock is already acquired by other thread, the current thread is blocked (until lock is released by the locking thread).
- "`synchronized`" non-static method acquires lock on the current object i.e. "`this`"

Click on the below Link for mysql Installation

<https://dev.mysql.com/downloads/installer/>

- Select the msi file shown below

Windows (x86, 32-bit), MSI Installer	8.0.40	306.5M	Download
(mysql-installer-community-8.0.40.0.msi)	MD5: 9e91b6b515ddea4495b62ffb7cee1f21 Signature		

SUNBEAM